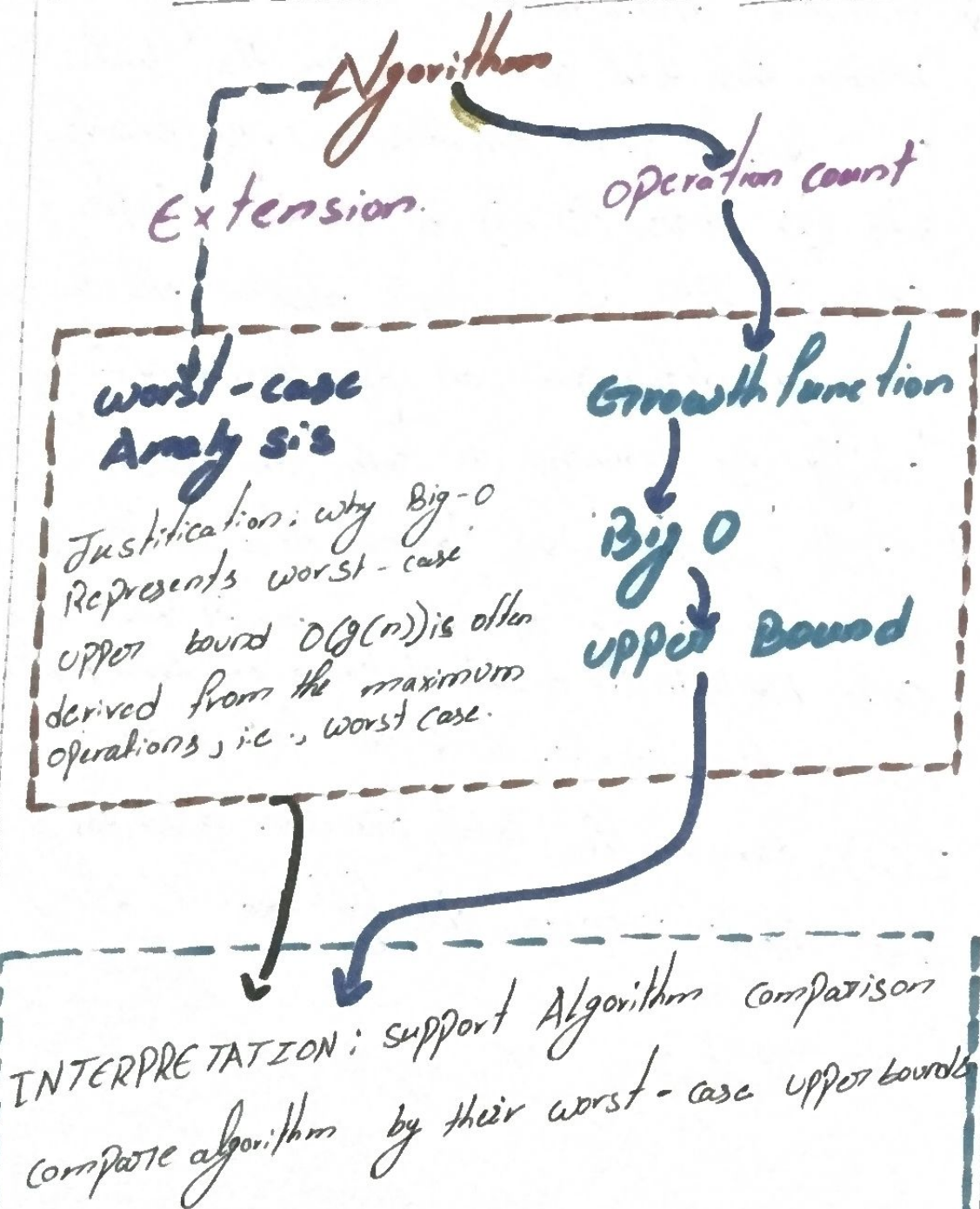
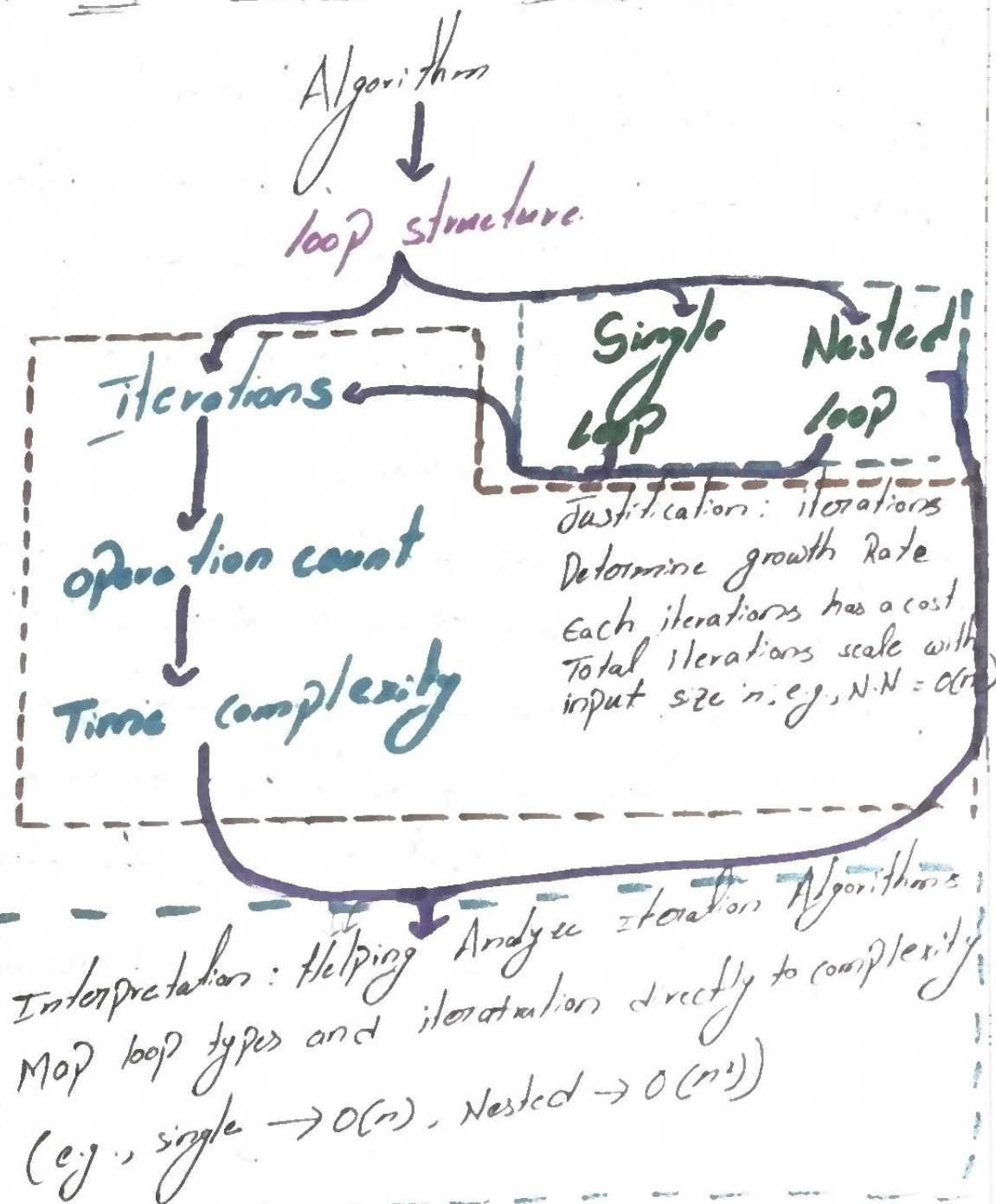


Name: V. Vignandhan
Reg no: 192472353
Sub code: CSA 0606

Q36: ASYMPTOTIC ANALYSIS & BIG-O



Q76: LOOP STRUCTURES & TIME COMPLEXITY



* Q36: (Asymptotic Analysis):

* Counting primitive operations defines a growth function, which Big-O simplifies into a worst-case upper bound. This creates hardware-independent safety ceiling, allowing developers to clearly compare different algorithms based on their maximum execution limits.

* The primitive operation Blueprint: when evaluating an algorithm, we treat every foundation step - such as an array lookup, a variable assignment, or an arithmetic evaluation - as a single unit of work costing " C_i " time. For a given input size " n ", the total time is expressed as an exact polynomial function.

$$T(n) = \sum C_i \cdot \text{Step}_i$$

Q76: (Loop structures)

* Iterative constructs drive time complexity because their total cost multiplies the iteration count by the inner operation cost. By branching into single loops ($O(n)$) or Nested loops ($O(n^2)$), this framework allows you to instantly determine an algorithm's efficiency simply by scanning its code structure.

* The multiplicative Nature of loops: Every loop acts as an operational multiplier. If a code block inside a loop takes a constant number of operations C , and the loop runs $I(n)$ times, the total operation count for that block scales directly as

$$T(n) = I(n) \times C$$